Mercredi · Travailler à plusieurs

Hier, M7 s'est terminé sur un refus : impossible de pousser dans le dépôt du voisin. Logique, GitHub ne laisse pas n'importe qui modifier votre code. Aujourd'hui, on apprend à travailler à plusieurs sur le même dépôt, proprement : chacun sur sa branche, et rien n'arrive dans `main` sans avoir été relu.

1. Deux personnes, un dépôt

Pour voir ce qui se passe, on simule une équipe sur une seule machine. Le dossier `github/equipe.git` joue le rôle de GitHub. Alice et Bob en ont chacun une copie (un *clone*), dans leurs dossiers `alice` et `bob`. Dans la vraie vie, chacun est sur son ordinateur et l'adresse est `git@github.com:...`, mais Git se comporte exactement pareil.

Mise en place : un dépôt partagé, deux clones

```
$ git clone github/equipe.git alice
Cloning into 'alice'...
done.
$ git clone github/equipe.git bob
Cloning into 'bob'...
done.
$ ls alice bob
alice:
README.md
bonjour.sh

bob:
README.md
bonjour.sh
```

Le push refusé

Alice modifie le README et pousse. Pendant ce temps, Bob modifie un autre fichier, commite, et pousse à son tour :

Alice pousse en premier

```
$ cd alice
$ echo "Équipe : Alice et Bob" >> README.md
$ git commit -qam "Ajoute les membres de l'équipe"
$ git push
To /home/robin/demo/github/equipe.git
   7ec32e6..d37c7c9  main -> main
```

Bob pousse ensuite… refusé

```
$ cd ../bob
$ echo 'echo "au revoir"' >> bonjour.sh
$ git commit -qam "Ajoute l'au revoir"
$ git push
To /home/robin/demo/github/equipe.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to '/home/robin/demo/github/equipe.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

Lisez le message : `Updates were rejected because the remote contains work that you do not have locally`. Le dépôt partagé contient le commit d'Alice, que Bob n'a pas. Git refuse que Bob l'écrase. La solution est indiquée juste en dessous : récupérer d'abord avec `git pull`.

Bob récupère, puis pousse

```
$ git pull
From /home/robin/demo/github/equipe
   7ec32e6..d37c7c9  main       -> origin/main
Merge made by the 'ort' strategy.
 README.md | 1 +
 1 file changed, 1 insertion(+)
$ git log --oneline --graph
*   5c9c792 (HEAD -> main) Merge branch 'main' of /home/robin/demo/github/equipe
|\
| * d37c7c9 (origin/main, origin/HEAD) Ajoute les membres de l'équipe
* | eb28c9b Ajoute l'au revoir
|/
* 7ec32e6 Initialise le projet
$ git push
To /home/robin/demo/github/equipe.git
   d37c7c9..5c9c792  main -> main
```

`git pull` a récupéré le commit d'Alice et l'a fusionné avec celui de Bob. Comme ils ne touchaient pas les mêmes fichiers, pas de conflit. Chez vous, Git ouvrira nano pour le message de fusion : gardez-le, enregistrez, fermez. S'ils avaient modifié la même ligne, vous auriez eu un conflit, à résoudre exactement comme hier.

La règle à retenir : **avant de commencer à travailler, `git pull`**. Moins vous avez de retard sur les autres, moins vous aurez de conflits.

`git pull`, c'est en fait deux commandes : `git fetch` (télécharger les nouveaux commits sans rien toucher) puis `git merge` (les fusionner). `git fetch` seul sert à regarder ce que les autres ont fait avant de fusionner.

2. La méthode de travail en équipe

Pousser tous sur `main`, ça marche à deux sur un petit projet. Au-delà, c'est le chaos : `main` casse régulièrement, et personne ne relit rien. La méthode utilisée dans la plupart des équipes tient en six étapes :

1. **Une tâche = une issue.** On décrit sur GitHub ce qu'il faut faire.
2. **Une issue = une branche.** On part d'un `main` à jour : `git switch main`, `git pull`, puis `git switch -c ajout-score`.
3. On commite sur la branche, et on la pousse : `git push -u origin ajout-score`.

4. On ouvre une **pull request** (PR) sur GitHub : « voici mes changements, je propose de les fusionner dans `main` ».
5. Un ou une collègue **relit**, commente, et finit par approuver.
6. On fusionne la PR sur GitHub, puis chacun met son `main` à jour.

`main` ne reçoit donc que du code relu, qui fonctionne.

Bob travaille sur une branche et la pousse

```
$ git switch -c ajout-date
Switched to a new branch 'ajout-date'
$ echo 'date "+Nous sommes le %d/%m/%Y"' >> bonjour.sh
$ git commit -qam "Affiche la date du jour"
$ git push -u origin ajout-date
To /home/robin/demo/github/equipe.git
 * [new branch]      ajout-date -> ajout-date
branch 'ajout-date' set up to track 'origin/ajout-date'.
```

Sur GitHub, la réponse du `push` contient en plus une ligne `remote: Create a pull request for 'ajout-date' on GitHub by visiting:` suivie d'un lien. Cliquez dessus, ou allez dans l'onglet **Pull requests** : GitHub propose un bouton **Compare & pull request**.

Une fois la PR fusionnée sur GitHub (ici, on la simule), Bob fait le ménage chez lui :

Après la fusion de la PR : mettre main à jour

```
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
$ git pull
From /home/robin/demo/github/equipe
   5c9c792..599860a  main       -> origin/main
Updating 5c9c792..599860a
Fast-forward
 bonjour.sh | 1 +
 1 file changed, 1 insertion(+)
$ git branch -d ajout-date
Deleted branch ajout-date (was 599860a).
$ git log --oneline --graph
* 599860a (HEAD -> main, origin/main, origin/ajout-date, origin/HEAD) Affiche la date du jour
*   5c9c792 Merge branch 'main' of /home/robin/demo/github/equipe
|\
| * d37c7c9 Ajoute les membres de l'équipe
* | eb28c9b Ajoute l'au revoir
|/
* 7ec32e6 Initialise le projet
```

3. La pull request

Une PR, c'est une demande de fusion accompagnée d'une conversation. Ce qui fait une bonne PR :

- **un titre clair**, comme un message de commit : « Ajoute l'affichage du score » ;
- **une description** qui répond à trois questions : qu'est-ce que ça change, pourquoi, comment le tester ;
- **une PR = une chose.** Une PR de 15 lignes est relue en 5 minutes. Une PR de 800 lignes n'est jamais vraiment relue.

Si la PR règle une issue, écrivez `Closes #12` dans la description (12 est le numéro de l'issue). GitHub fermera l'issue automatiquement quand la PR sera fusionnée.

```
Titre : Ajoute la commande « stats »

Ce que ça change :
- nouvelle commande ./todo.sh stats
- affiche le nombre de tâches faites et restantes

Pour tester :
./todo.sh ajouter "test"
./todo.sh stats   → doit afficher « 0 faite / 1 restante »

Closes #7
```

4. Relire le code des autres

Dans la PR, l'onglet **Files changed** montre le `diff`. Survolez une ligne, cliquez sur le **+** bleu : vous pouvez commenter cette ligne précise. À la fin, bouton **Review changes**, puis trois choix :

Choix	Quand l'utiliser
Comment	une remarque, sans donner d'avis final
Approve	c'est bon pour moi, on peut fusionner
Request changes	il faut corriger quelque chose avant de fusionner

Une relecture, c'est d'abord récupérer la branche et **lancer le code** (`git fetch`, puis `git switch nom-de-la-branche`). Lire sans tester, c'est relire à moitié.

Un commentaire de relecture vise le code, pas la personne. Il est précis, et propose une solution :

Pas utile	Utile
« C'est faux. »	« Avec `./todo.sh terminer 99`, le script plante : il faudrait vérifier que la tâche 99 existe. »
« Pas terrible, ce nom. »	« `t` n'est pas très parlant, `tache` serait plus clair ? »
« LGTM » sans avoir rien lancé	« Testé avec 3 tâches, tout fonctionne. J'approuve. »

5. Les issues

L'onglet **Issues** sert à lister ce qu'il y a à faire : des fonctionnalités, des bugs, des questions. Une bonne issue a un titre court, une description (et pour un bug : comment le reproduire), un ou une responsable (*Assignees*) et des étiquettes (*Labels*, par exemple `bug` ou `enhancement`). Chaque issue a un numéro, `#1`, `#2`… que l'on cite dans les commits et les PR.

Jeudi, les issues deviendront des user stories, et elles alimenteront le tableau de bord de votre équipe.

6. Régler son dépôt d'équipe

- **Inviter l'équipe** : dans le dépôt, **Settings**, **Collaborators**, **Add people**. Chaque invité reçoit un mail et doit accepter l'invitation, sinon il ne pourra pas pousser.
- **Protéger `main`** : **Settings**, **Branches** (ou **Rules**, selon l'interface), ajoutez une règle sur `main` et cochez l'obligation de passer par une pull request. Plus personne ne pourra pousser directement sur `main`, même par erreur.

Exercices du mercredi

Ces exercices se font **en équipe de 3**. Désignez une personne pour créer le dépôt, les autres le cloneront.

Me1. Le dépôt d'équipe NIVEAU 1

1. Une personne crée sur GitHub un dépôt `equipe-<nom>`, **en cochant** « Add a README file » cette fois (il servira de point de départ).
2. Elle invite les deux autres comme collaborateurs.
3. Les invités acceptent l'invitation reçue par mail, puis tout le monde clone le dépôt.
4. Chacun vérifie avec `git log --oneline` qu'il voit le commit initial.

Me2. Chacun sa branche, chacun sa PR NIVEAU 1

1. Chacun crée une branche `ajout-<prenom>` et y ajoute **un** de ses scripts bash de la semaine dernière.
2. Chacun pousse sa branche et ouvre une pull request avec un titre et une description qui explique comment lancer le script.
3. Chaque PR est relue par une autre personne de l'équipe, qui teste le script et laisse au moins un commentaire, puis approuve.
4. L'auteur fusionne sa PR. Tout le monde finit avec `git switch main` et `git pull`, et doit voir les trois scripts.

Me3. Le push refusé NIVEAU 2

Pour une fois, **deux personnes travaillent directement sur `main`**, sur des fichiers différents.

1. Chacune modifie un fichier différent et commite.
2. La première pousse : ça passe.
3. La seconde pousse : refusé. Lisez le message ensemble.
4. Réglez la situation avec `git pull`, puis `git push`.

Me4. Le conflit à deux NIVEAU 2

1. Deux personnes créent chacune une branche et modifient **la même ligne** du README (la première ligne, par exemple). Elles poussent et ouvrent chacune une PR.
2. Fusionnez la première PR. Sur la seconde, GitHub affiche : *This branch has conflicts that must be resolved*.
3. L'auteur de la seconde PR règle le conflit sur sa machine :

```
git switch sa-branche
git pull origin main
```

1. Résolvez le conflit comme mardi, commitez, puis `git push`. La PR redevient fusionnable.

Me5. Relecture de code NIVEAU 2

Une personne de l'équipe ajoute ce script dans une branche et ouvre une PR. Il calcule une moyenne et la meilleure note… et il contient des bugs.

moyenne_a_relire.sh

```
#!/bin/bash
# Affiche la moyenne et la meilleure des notes passées en arguments
notes=($@)
for n in "${notes[@]}"; do
    somme=$(( somme + n ))
done
moyenne=$(( somme / 5 ))
echo "Moyenne : $moyenne"
echo "Meilleure note : ${notes[0]}"
```

Les deux autres relisent la PR. Lancez le script avec `bash moyenne_a_relire.sh 12 8 15`, trouvez les problèmes, et laissez **un commentaire par problème** sur la bonne ligne, en proposant une correction. Terminez par **Request changes**. L'auteur corrige dans la même branche, pousse (la PR se met à jour toute seule), et les relecteurs approuvent.

Me6. Issue fermée automatiquement NIVEAU 2

1. Créez trois issues dans votre dépôt, par exemple : « Ajouter un README détaillé », « Ajouter un script de compte à rebours », « Corriger la moyenne ». Assignez chacune à quelqu'un.
2. Chacun traite son issue dans une branche, et met `Closes #<numéro>` dans la description de sa PR.
3. Après la fusion, vérifiez dans l'onglet Issues que l'issue s'est fermée toute seule.

Me7. Protéger main NIVEAU 3

Le propriétaire du dépôt active la protection de `main` (section 6). Essayez ensuite de pousser un commit directement sur `main` : quel message obtenez-vous ?

Mémo Git

Besoin	Commande
Créer un dépôt dans le dossier courant	`git init`
Où j'en suis ?	`git status` (version courte : `git status -s`)
Ajouter des fichiers au prochain commit	`git add fichier`, ou `git add .` pour tout
Enregistrer un commit	`git commit -m "Message au présent"`
Ajouter et commiter les fichiers déjà suivis	`git commit -am "Message"`
Voir ce qui a changé (pas encore ajouté)	`git diff`
Voir ce qui va partir dans le commit	`git diff --staged`
Historique	`git log --oneline`, `git log --oneline --graph --all`
Détail d'un commit	`git show <id>`
Annuler une modification non ajoutée	`git restore fichier` (perd la modification)
Sortir un fichier de l'index	`git restore --staged fichier`
Annuler un commit sans réécrire l'historique	`git revert <id>`
Lister les branches	`git branch`
Créer une branche et y aller	`git switch -c nom`
Changer de branche	`git switch nom`
Fusionner une branche dans la branche courante	`git merge nom`
Supprimer une branche fusionnée	`git branch -d nom`
Abandonner une fusion en conflit	`git merge --abort`
Relier le dépôt à GitHub	`git remote add origin git@github.com:pseudo/depot.git`
Premier envoi d'une branche	`git push -u origin nom`
Envoyer / récupérer	`git push` / `git pull`
Télécharger un dépôt	`git clone <adresse>`
Récupérer sans fusionner	`git fetch`
Qui a écrit cette ligne ?	`git blame fichier`
Quel commit a ajouté ce texte ?	`git log -S "texte" --oneline`

Dans un conflit, le fichier contient `<<<<<<< HEAD` (votre version), `=======`, puis `>>>>>>> branche` (l'autre version). Gardez ce que vous voulez, supprimez les trois marqueurs, puis `git add` et `git commit`.